

PRAKTIKUM BASIS DATA
MODUL 11
NoSQL – Key Value Database



Oleh:
Putra Anugrah Pamungkas 103122400007

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO UNIVERSITAS
TELKOM
2026

1. Davviz adalah seorang Backend Engineer handal gg jago yang sedang membangun platform streaming musik generasi terbaru bernama Spotifine. Aplikasi ini akan memiliki jutaan pengguna aktif, sehingga Davviz membutuhkan database yang tahan banting, dan tidak akan mati meskipun salah satu servernya mengalami gangguan. Davviz ditugaskan untuk merancang database yang menyimpan Data Musisi di dalam platform Spotifine. Karena satu musisi bisa memiliki banyak genre dan pengikut dari berbagai negara, Davviz memilih Cassandra sebagai solusinya. Berikut atribut yang perlu dibuat :
 - a. Artist ID: ID unik untuk setiap musisi (Primary Key).
 - b. Artist Name: Nama lengkap musisi atau nama panggung
 - c. Followers: Jumlah pengikut (Tipe data int).
 - d. Origin: Negara asal musisi.
 - e. Genres: Daftar aliran musik yang diusung (Gunakan tipe data set karena satu musisi bisa memiliki genre lebih dari satu, misal: Pop dan R&B).

Jawaban:

1. **Artist ID:** text (Primary Key)
 2. **Artist Name:** text
 3. **Followers:** int
 4. **Origin:** text
 5. **Genres:** set<text> (Kumpulan aliran musik)
2. Implementasi
 - a. Buat Keyspace
 - buatlah sebuah Keyspace dengan nama spotifine_0801 sebagai wadah utama data aplikasi
 - Gunakan strategi replikasi SimpleStrategy dengan replication_factor
 - Tuliskan query-nya dan sertakan Screenshot saat keyspace berhasil dibuat dan dipanggil menggunakan perintah USE

b. Buatlah table

- Buatlah tabel bernama musician_data di dalam keyspace spotifine_0801 dengan atribut yang sudah direncanakan Davviz.
- Tuliskan query CREATE TABLE sesuai atribut di atas dan sertakan Screenshot.
- Masukkan (Insert) minimal 3 data musisi berbeda ke dalam tabel. (Pastikan salah satu musisi memiliki lebih dari satu genre dalam kolom set).
- Tampilkan semua data yang telah dimasukkan menggunakan perintah SELECT.
- Sertakan Screenshot hasil tabel akhir yang sudah terisi data.

Jawaban :

a.

b. Keyspace

```
cqlsh> CREATE KEYSPACE spotifine_0801
... WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
cqlsh>
cqlsh> USE spotifine_0801;|

cqlsh> USE spotifine_0801;CREATE TABLE musician_data (
...     artist_id int PRIMARY KEY,
...     artist_name text,
...     followers int,
...     origin text,
...     genres set<text>
... );
```

c. Buat Table

```
cqlsh:spotifine_0801> INSERT INTO musician_data (artist_id, artist_name, followers, origin, genres)
... VALUES (1, 'Tulus', 5000000, 'Indonesia', {'Pop', 'Jazz'});
cqlsh:spotifine_0801>
cqlsh:spotifine_0801> INSERT INTO musician_data (artist_id, artist_name, followers, origin, genres)
... VALUES (2, 'The Weeknd', 80000000, 'Canada', {'R&B', 'Pop', 'Synth-pop'});
cqlsh:spotifine_0801>
cqlsh:spotifine_0801> INSERT INTO musician_data (artist_id, artist_name, followers, origin, genres)
... VALUES (3, 'Rich Brian', 4000000, 'Indonesia', {'Hip Hop', 'Rap'});
cqlsh:spotifine_0801>
```

```
cqlsh:spotifine_0801> SELECT * FROM musician_data;
```

artist_id	artist_name	followers	genres	origin
1	Tulus	5000000	{'Jazz', 'Pop'}	Indonesia
2	The Weeknd	80000000	{'Pop', 'R&B', 'Synth-pop'}	Canada
3	Rich Brian	4000000	{'Hip Hop', 'Rap'}	Indonesia

(3 rows)
cqlsh:spotifine_0801>

3. Apa keuntungan utama menggunakan tipe data set pada kolom genres di aplikasi Spotifine dibandingkan jika ia menggunakan tabel relasional (RDBMS) biasa?

Jawaban :

Keuntungan utamanya adalah peningkatan kecepatan waktu respon saat membaca data (read operations) dengan meniadakan proses JOIN antar tabel. Pada RDBMS konvensional, satu musisi yang memiliki banyak genre mengharuskan pembuatan tabel terpisah (tabel relasi) untuk menghindari redundansi. Untuk menampilkan data musisi beserta genrenya, sistem RDBMS harus melakukan proses JOIN yang mengkonsumsi beban komputasi tinggi, terlebih jika Spotifine memiliki jutaan pengguna aktif. Dengan Cassandra, tipe data set memungkinkan banyak genre disimpan secara langsung pada satu kolom (denormalized). Server tidak perlu lagi menggabungkan tabel, sehingga pengambilan dan pencarian data menjadi jauh lebih efisien, cepat, serta tahan banting.